

27.10.2025

Алгоритмы «Разделяй и властвуй». Строковые алгоритмы.

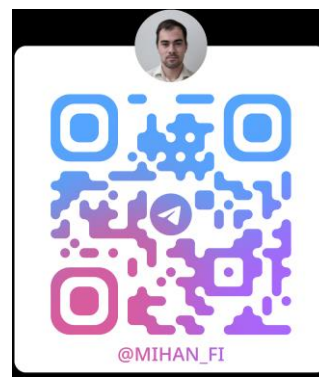
Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

N * Новосибирский
государственный
университет
***НАСТОЯЩАЯ НАУКА**

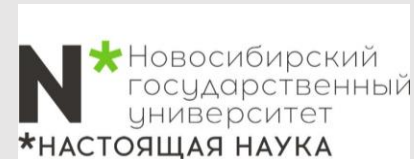


Давайте познакомимся



Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Backend C++ разработчик в Яндекс.
- м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++



Адженда

**Алгоритмы
«Разделяй и
властвуй»**

30 минут

**Строковые
алгоритмы**

60 минут

Адженда

**Алгоритмы
«Разделяй и
властвуй»**

30 минут

**Строковые
алгоритмы**

60 минут

Введение

«Разделяй и властвуй» - класс алгоритмических методов, которые разбивают входные данные на несколько частей, рекурсивно решают задачу для каждой части, а затем объединяют решения подзадач в одно общее решение.

Анализ времени выполнения алгоритма категории **«разделяй и властвуй»** обычно подразумевает вычисление рекуррентного соотношения, которое устанавливает рекурсивную границу времени выполнения в контексте времени выполнения меньших экземпляров. У многих ситуаций, в которых применяется принцип «разделяй и властвуй», есть нечто общее: естественный алгоритм «грубой силы» может выполняться за полиномиальное время, а стратегия «разделяй и властвуй» способна сократить время выполнения до многочлена меньшей степени.

Примеры:

- Бинарный поиск
- Сортировка слиянием (след. лекция)
- Подсчет инверсий
- Поиск ближайшей пары точек
- Целочисленное умножение и деление
- Свертки и быстрое преобразование Фурье



Задача 1 – бинарный поиск

Дан отсортированный массив. Необходимо найти произвольное число.

Вывести номер элемента массива или -1 при отсутствии числа в массиве.

При сравнении целевого значения со средним элементом отсортированного списка возможен один из трех результатов: значения равны, целевое значение меньше элемента списка, либо целевое значение больше элемента списка. В первом, и наилучшем, случае поиск завершен. В остальных двух случаях мы можем отбросить половину списка.

```
int binarySearch(int arr[], int left, int right, int target) {  
    if (right >= left) {  
        int mid = left + (right - left) / 2;  
        if (arr[mid] == target) {  
            return mid;  
        }  
        if (arr[mid] > target) {  
            return binarySearch(arr, left, mid - 1, target);  
        }  
        return binarySearch(arr, mid + 1, right, target);  
    }  
    return -1;  
}
```



Цикл - бинарный поиск

Однако, если есть возможность просто перейти от рекурсивного алгоритма к алгоритму с циклом – делаем это следующим этапом.

```
int binarySearch(int arr[], int n, int target) {  
    int left = 0;  
    int right = n - 1;  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
        if (arr[mid] == target) {  
            return mid;  
        }  
        if (arr[mid] > target) {  
            right = mid - 1;  
        } else {  
            left = mid + 1;  
        }  
    }  
    return -1;  
}
```



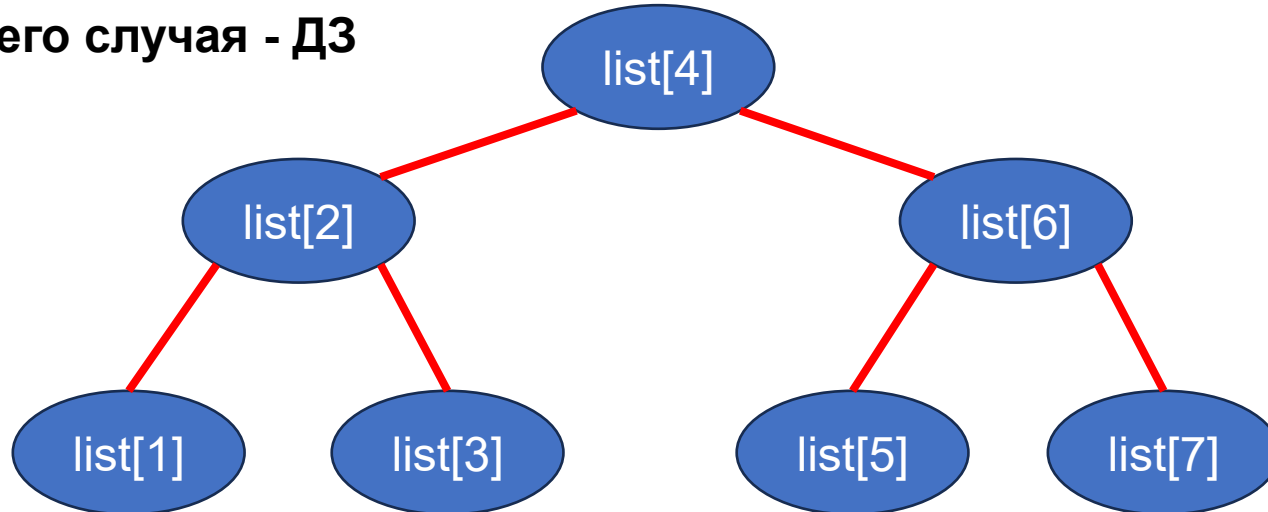
Анализ наихудшего случая

Степень двойки в длине оставшейся части списка при всяком проходе цикла уменьшается на единицу.

Последняя итерация цикла производится, когда размер оставшейся части становится равным 1. Последний проход $\text{left}=\text{right}$. Это означает, что при $N = 2^k - 1$ число проходов не превышает k . Решая последнее уравнение относительно k , мы заключаем, что в наихудшем случае число проходов равно $k = \log_2(N + 1)$.

В анализе может помочь и дерево решений для процесса поиска. Поскольку мы предполагаем, что $N = 2^k - 1$, соответствующее дерево решений всегда будет полным. В нем будет k уровней, где $k = \log_2(N + 1)$. Мы делаем по одному сравнению на каждом уровне, поэтому полное число сравнений не превосходит $\log_2(N + 1)$.

Анализ среднего случая - Д3



Задача 2 - Подсчет инверсий

Следующая задача встречается при анализе ранжирования. **Примеры использования:**

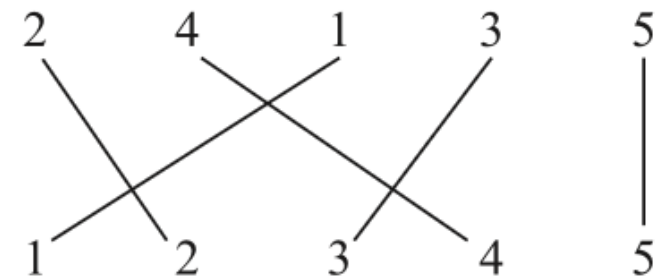
1. некоторые сайты применяют метод, называемый совместной фильтрацией, чтобы сопоставить ваши предпочтения (книги, кино, рестораны) с предпочтениями других людей в Интернете.
2. встречается в инструментах метапоиска, которые выполняют один запрос в разных поисковых системах, а потом пытаются синтезировать результаты на основании сходств и различий между рангами, полученными от разных поисковых систем.

Рассмотрим задачу сравнения двух ранжировок набора фильмов: вашей и чьей-то еще. Естественный метод — пометить фильмы от 1 до n в соответствии с вашей ранжировкой, затем упорядочить эти метки в соответствии с ранжировкой другого человека и посмотреть, сколько пар «нарушает порядок».

Формулировка задачи: имеется последовательность из n чисел $a_1, \dots, a_n$; предполагается, что все числа различны. Требуется определить метрику, которая сообщает, насколько список отклоняется от упорядочения по возрастанию; значение метрики должно быть равно 0, если $a_1 < a_2 < \dots < a_n$, и должно быть тем выше, чем сильнее нарушен порядок чисел.

Естественное количественное выражение этого понятия основано на подсчете инверсий. Мы будем говорить, что два индекса $i < j$ образуют инверсию, если $a_i > a_j$, то есть если два элемента a_i и a_j идут «не по порядку». Нужно определить количество инверсий в последовательности $a_1, \dots, a_n$.

Пример: 2, 4, 1, 3, 5. В нем присутствуют три инверсии: (2, 1), (4, 1) и (4, 3).



Разработка и анализ алгоритма

Алгоритм «грубой силы» $O(n^2)$.

Мы хотим за время $O(n \log n)$. Примерный набросок:

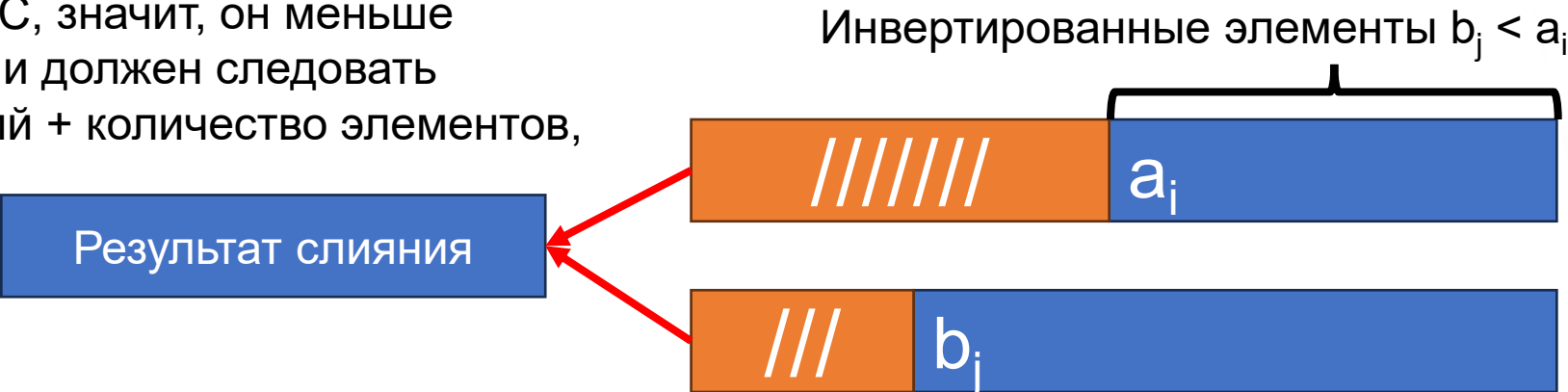
1. Делим список на две части: $a_1, \dots, a_m$ и $a_{m+1}, \dots, a_n$.
2. Количество инверсий в каждой половине по отдельности сколько?
3. Инверсии (a_i, a_j) для двух чисел, принадлежащих разным половинам; хитрость в том, что это нужно сделать за время $O(n)$.

Дополнения которые могут ускорить алгоритм:

1. Рекурсивная сортировка чисел в двух половинах.
2. «Объединяющий» шаг для сортировки слиянием: $O(n)$ + подсчитать количество «инвертированных пар» (a, b) , для которых $a \in A$, $b \in B$, и $a > b$.

Как подсчитать количество инверсий? Списки A и B отсортированы, сливаем в C с условием:

- a_i добавляется к C, новые инверсии не возникают $\rightarrow a_i$ меньше всех элементов, оставшихся в списке B.
- b_j присоединяется к списку C, значит, он меньше всех оставшихся элементов A и должен следовать всех, поэтому счетчик инверсий + количество элементов, оставшихся в A.



Разработка и анализ алгоритма

Merge-and-Count(A,B)

Хранить для каждого списка указатель Current, инициализируемый указателем на начальный элемент

Хранить количество инверсий в переменной Count, инициализируемой 0

Пока оба списка не пусты:

 Пусть a_i и b_j — элементы, на которые указывают указатели Current

 Присоединить меньший из них к выходному списку

 Если меньшим является элемент b_j , то

 Увеличить Count на количество элементов, оставшихся в A

 Конец Если

 Переместить указатель

 Current в списке, из которого был выбран меньший элемент.

Конец Пока

Когда один из списков опустеет, присоединить остаток другого списка к выходному списку

Вернуть Count и объединенный список

Разработка и анализ алгоритма

Sort-and-Count(L)

Если список содержит один элемент, инверсий нет

Иначе

Разделить список на две половины:

A содержит первые $n/2$ элементов

B содержит остальные $n/2$ элементов

$(r_A, A) = \text{Sort-and-Count}(A)$

$(r_B, B) = \text{Sort-and-Count}(B)$

$(r, L) = \text{Merge-and-Count}(A, B)$

Конец Если

Вернуть $r = r_A + r_B + r$ и отсортированный список L

Так как слияние с подсчетом выполняется за время $O(n)$, время выполнения $T(n)$ полной процедуры сортировки с подсчетом удовлетворяет рекуррентному отношению:

Для некоторой константы c , $T(n) \leq 2T(n/2) + cn$ для $n > 2$, и $T(2) \leq c$.

Алгоритм сортировки с подсчетом i правильно сортирует входной список и подсчитывает количество инверсий; для списка с n элементами он выполняется за время $O(n \log n)$.

Задача 3 - Поиск ближайшей пары точек

Формулировка задачи очень проста: для заданных n точек на плоскости найти пару точек, расположенных ближе друг к другу.

В начале 1970-х годов эту задачу рассматривали М. И. Шамос и Д. Хоуи в ходе проекта по поиску эффективных алгоритмов для базовых вычислительных примитивов для геометрических задач. Эти алгоритмы заложили основу зарождающейся в то время области вычислительной геометрии и проникли в такие области, как компьютерная графика, обработка изображений, географические информационные системы и молекулярное моделирование. И хотя задача нахождения ближайшей пары точек принадлежит к числу самых естественных алгоритмических задач в геометрии, найти для нее эффективный алгоритм оказывается на удивление сложно. Очевидно, существует решение $O(n^2)$ — вычислить расстояния между каждой парой точек и выбрать минимум; Шамос и Хоуи задались вопросом, существует ли алгоритм, который был бы асимптотически более быстрым, чем квадратичный. Прошло довольно много времени, прежде чем был получен ответ на этот вопрос. Приведенный ниже алгоритм $O(n \log n)$ по сути не отличается от найденного ими. Более того, когда мы вернемся к этой задаче в конце семестра, то увидим, что время выполнения можно дополнительно улучшить до $O(n)$ за счет применения рандомизации.

Разработка алгоритма

Определения некоторых обозначений.

Множество точек $P = \{p_1, \dots, p_n\}$, в котором точка p_i имеет координаты (x_i, y_i) ; $p_i, p_j \in P$ стандартное евклидово расстояние между ними обозначим $d(p_i, p_j)$.

Цель: найти пару точек p_i, p_j , минимизирующую $d(p_i, p_j)$.

Упрощение разработки: никакие две точки из P не имеют одинаковых значений координаты x или y .

Одномерная версия этой задач. Как найти ближайшую пару точек на прямой?

- Отсортировать точки за время $O(n \log n)$,
- Перебрать отсортированный список с вычислением расстояния от каждой точки до следующей $O(n)$.

Одномерное решение к двум измерениям не приспособить!

Стратегия в стиле «разделяй и властвуй»

- мы находим ближайшую пару среди точек в «левой половине» P и ближайшую пару среди точек в «правой половине» P
- эта информация используется для получения общего решения за линейное время. Если разработать алгоритм с такой структурой, то решение $O(n \log n)$.



Подготовка рекурсии

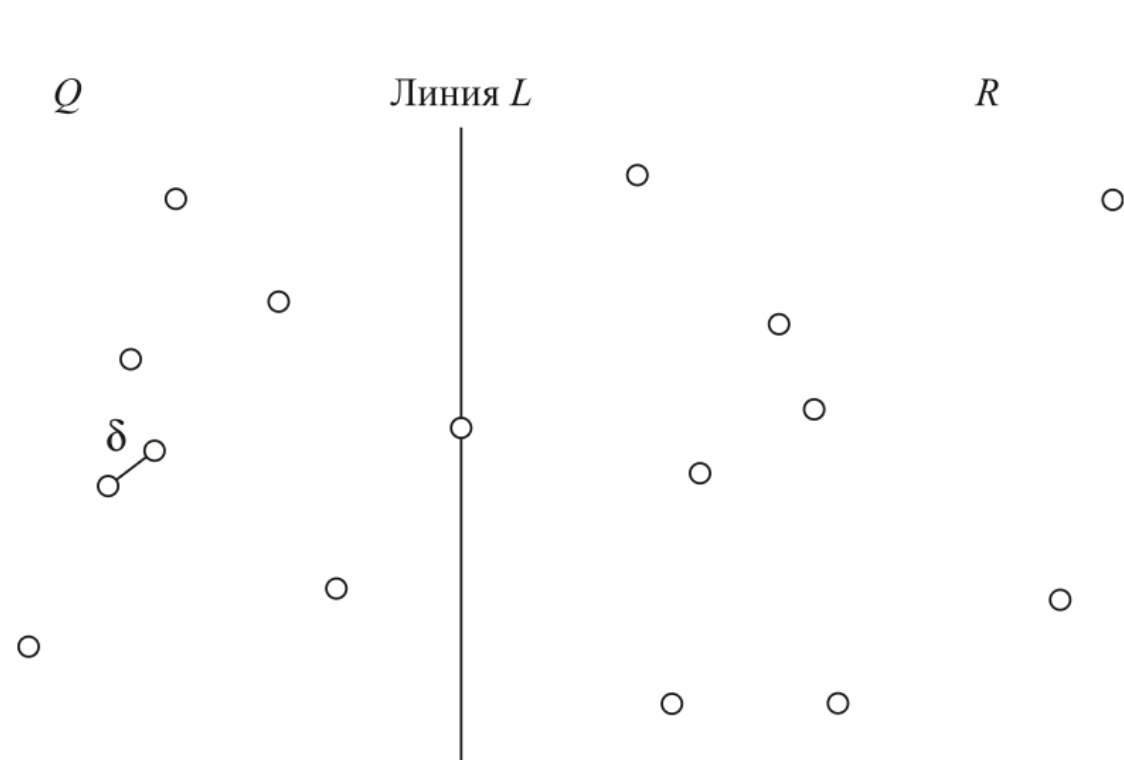
Каждый рекурсивный вызов для множества $P' \subseteq P$ начинался с двух списков:

P'_x , в котором все точки P' отсортированы по возрастанию координаты x ,

P'_y , в котором все точки P' отсортированы по возрастанию координаты y .

Алгоритм

- все точки P сортируются по координате x , а потом по координате y ; так получаются списки P_x и P_y .
- Первый уровень рекурсии работает так, как описано ниже; другие уровни работают полностью аналогично.



1. Q определяется как множество точек в первых $[n/2]$ P_x («левая половина»), а R — («правая половина»). За один проход по P_x и P_y за время $O(n)$ можно создать четыре списка: Q_x с точками Q, отсортированными по возрастанию координаты x ; Q_y с точками Q, по y ; и два аналогичных списка R_x и R_y . Для каждого элемента каждого из списков также хранится его позиция в обоих исходных списках.
2. Рекурсивно определяется ближайшая пара точек в Q (с обращениями к спискам Q_x и Q_y). Предположим, (правильно) возвращены как ближайшая пара точек в Q. Аналогичным образом определяется ближайшая пара точек q_0^* и q_1^* в R, обозначим их r_0^* и r_1^* .

Объединение решений

Как использовать решения двух подзадач в «объединяющей» операции с линейным временем?

Обозначение δ для минимума из $d(q_0^*, q_1^*)$ и $d(r_0^*, r_1^*)$.

? существуют ли точки $q \in Q$ и $r \in R$, для которых $d(q, r) < \delta$? Если нет, то ближайшая пара уже была найдена одним из предшествующих рекурсивных вызовов. Если же такие точки существуют, то ближайшие q и r образуют ближайшую пару в P .

Обозначим:

x^* обозначает координату x крайней правой точки в Q ,

L — вертикальную линию, описываемую уравнением $x = x^*$.

Лемма 9.1. Если существуют $q \in Q$ и $r \in R$, для которых $d(q, r) < \delta$, то каждая из точек q и r располагается в пределах расстояния δ от L .

Доказательство. Предположим, такие q и r существуют; обозначим $q = (q_x, q_y)$ и $r = (r_x, r_y)$. Из определения x^* следует, что $q_x \leq x^* \leq r_x$. Тогда

$$x^* - q_x \leq r_x - q_x \leq d(q, r) < \delta$$

$$r_x - x^* \leq r_x - q_x \leq d(q, r) < \delta,$$

а это значит, что у каждой из точек q и r координата x находится в пределах δ от x^* — и следовательно, ее расстояние от линии L не превышает δ . ■

Множество точек в пределах расстояния δ от L обозначим $S \subseteq P$.

Следствие 9.1. Существуют $q \in Q$ и $r \in R$, для которых $d(q, r) < \delta$ в том и только в том случае, если существуют $s, s' \in S$, для которых $d(s, s') < \delta$.



Объединение решений

Каждое поле
может содержать
не более одной
входной точки

Лемма 9.2. Если $s, s' \in S$ обладают тем свойством, что $d(s, s') < \delta$, то s и s' находятся на расстоянии не более 15 позиций друг от друга в отсортированном списке S_y .

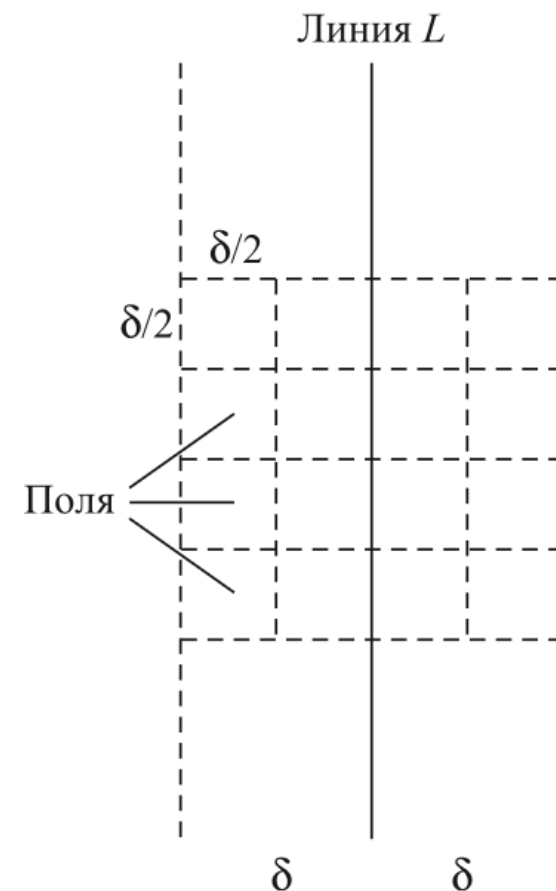
Доказательство. Рассмотрим подмножество Z плоскости, состоящее из всех точек, удаленных от L на расстояние не более δ . Разделим Z на поля — квадраты со стороной $\delta/2$. Один ряд Z будет состоять из четырех полей, горизонтальные стороны которых имеют одинаковые координаты y .

Допустим, две точки S лежат в одном поле. Так как все точки этого поля находятся по одну сторону от L , эти две точки либо обе принадлежат Q , либо обе принадлежат R .

Но любые две точки в одном поле находятся на расстоянии не более $\frac{\delta\sqrt{2}}{2} < \delta$, а это противоречит нашему определению δ как минимального более расстояния между любой парой точек Q или R . Следовательно, каждое поле содержит не более одной точки S .

Теперь допустим, что $s, s' \in S$ обладают свойством $d(s, s') < \delta$ и находятся на расстоянии 16 и более позиций в S_y . Предположим без потери общности, что s имеет меньшую координату y . Так как поле может содержать не более одной точки, между s и s' лежат как минимум три ряда Z . Но любые две точки в Z , разделенные минимум тремя рядами, должны находиться на расстоянии минимум $3\delta/2$ — противоречие. ■

Алгоритм: (i) ближайшую пару точек в S , если расстояние между ними меньше δ ; или (ii) (обоснованное) заключение о том, что в S не существует пар точек, разделенных расстоянием не более δ .



Краткое описание алгоритма

Closest-Pair(P)

Построить P_x и P_y (время $O(n \log n)$) (

$$(p_0^*, p_1^*) = \text{Closest-Pair-Rec}(P_x, P_y)$$

Closest-Pair-Rec(P_x, P_y)

Если $|P| \leq 3$

Найти ближайшую пару вычислением всех попарных расстояний
Конец Если

Построить Q_x, Q_y, R_x, R_y (время $O(n)$)

$$(q_0^*, q_1^*) = \text{Closest-Pair-Rec}(Q_x, Q_y)$$

$$(r_0^*, r_1^*) = \text{Closest-Pair-Rec}(R_x, R_y)$$

Краткое описание алгоритма

$$\delta = \min(d(q_0^*, q_1^*), (r_0^*, r_1^*))$$

x^* = максимальная координата x точки в множестве Q

$$L = \{(x, y) : x = x^*\}$$

S = точки в P , находящиеся от L на расстоянии не более δ .

Построить S_y (время $O(n)$)

Для каждой точки $s \in S_y$ вычислить расстояние от s до каждой из следующих 15 точек в S_y

Пусть s, s' — пара с минимальным расстоянием (время $O(n)$)

Если $d(s, s') < \delta$

Вернуть (s, s')

Иначе Если $d(q_0^*, q_1^*) < (r_0^*, r_1^*)$,

Вернуть (q_0^*, q_1^*)

Иначе

Вернуть (r_0^*, r_1^*)

Конец Если

Анализ алгоритма

Лемма 9.3. Алгоритм правильно находит ближайшую пару точек в P .

Доказательство.

Для доказательства правильности будет использована индукция по размеру P ; случай $|P| \leq 3$ очевиден. Для заданного P ближайшая пара рекурсивных вызовов правильно вычисляется индукцией. Из лемм 9.1 и 9.2 оставшаяся часть алгоритма правильно определяет, находится ли любая пара точек S на расстоянии менее δ , и если находится — возвращает такую пару с минимальным расстоянием. Теперь у ближайшей пары P либо обе точки принадлежат одному из множеств Q или R , либо они принадлежат разным множествам. В первом случае ближайшая пара находится рекурсивным вызовом; во втором она находится на расстоянии менее δ и ищется оставшейся частью алгоритма. ■

Лемма 9.4. Время выполнения алгоритма равно $O(n \log n)$.

Доказательство.

Исходная сортировка P по x и y выполняется за время $O(n \log n)$. Время выполнения оставшейся части алгоритма удовлетворяет рекуррентному отношению $T(n) \leq 2T(n/2) + cn$ для $n > 2$, и $T(2) \leq c$, а следовательно, равно $O(n \log n)$. ■



Адженда

**Алгоритмы
«Разделяй и
властвуй»**

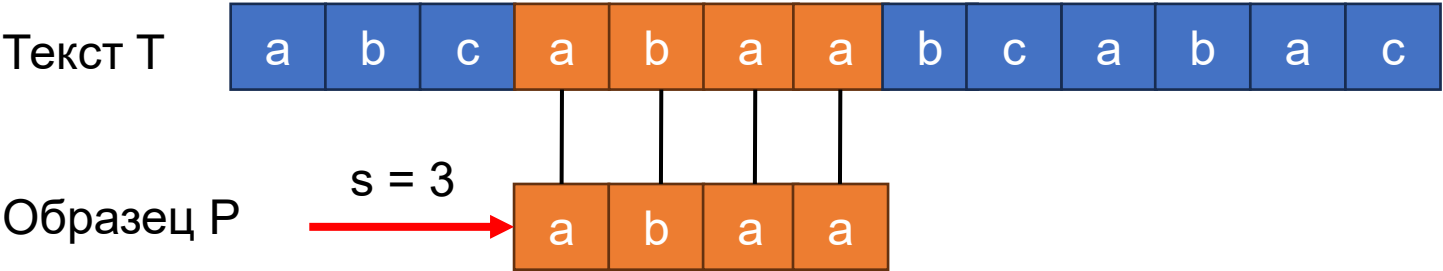
30 минут

**Строковые
алгоритмы**

60 минут

Введение

В программах, предназначенных для редактирования текста, часто необходимо найти все фрагменты текста, которые совпадают с заданным образцом. Обычно **текст** — это редактируемый документ, а **образец** — это искомое слово, введенное пользователем. Эффективные алгоритмы решения этой задачи (часто именуемой сопоставлением строк (string matching)) могут сокращать время реакции текстовых редакторов на действия пользователя. Среди множества приложений алгоритмы поиска подстрок используются, например, для поиска заданных образцов в молекулах ДНК. Поисковые системы в Интернете также используют их при поиске веб-страниц, отвечающих запросам.



Постановку задачи поиска подстрок (string-matching problem):

Дано:
Исходный текст $T[1...n]$
образец (шаблон) $P[1...m]$
 $m \leq n$.
Элементы массивов P и T - символы из конечного алфавита Σ .
Например, алфавит может иметь вид $\Sigma = \{0,1\}$ или $\Sigma = \{a,b,..., z\}$. Символьные массивы P и T часто называют строками (strings) символов.

Алгоритм	Время предварительной обработки	Время сравнения
"В лоб"	0	$O((n - m + 1)m)$
Рабина Карпа	$\Theta(m)$	$O((n - m + 1)m)$
Конечный автомат	$O(m \Sigma)$	$\Theta(n)$
Кнута-Морриса-Пратта	$\Theta(m)$	$\Theta(n)$

Обозначения и терминология

Σ^* множество всех строк конечной длины, образованных с помощью символов алфавита Σ .

Пустая строка (empty string) $\varepsilon \in \Sigma^*$.

Длина строки x обозначается $|x|$. $|\varepsilon| = 0$

Конкатенация (concatenation) строк x и y , которая обозначается xy , имеет длину $|x| + |y|$ (выполнение функции concat).

Строка w является **префиксом** (prefix) строки x ($w \sqsubset x$), если $x = wy$ для некоторой строки $y \in \Sigma^*$. Заметим, что если $w \sqsubset x$, то $|w| \leq |x|$.

Строку w называют **суффиксом** (suffix) строки x ($w \sqsupset x$), если существует такая строка $y \in \Sigma^*$, что $x = yw$. Как и в случае префикса, из $w \sqsupset x$ следует неравенство $|w| \leq |x|$.

Например, $ab \sqsubset abcca$ и $cca \sqsupset abcca$.

Факты:

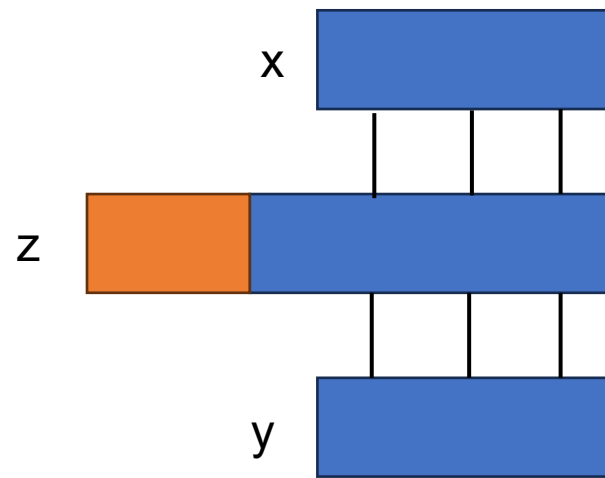
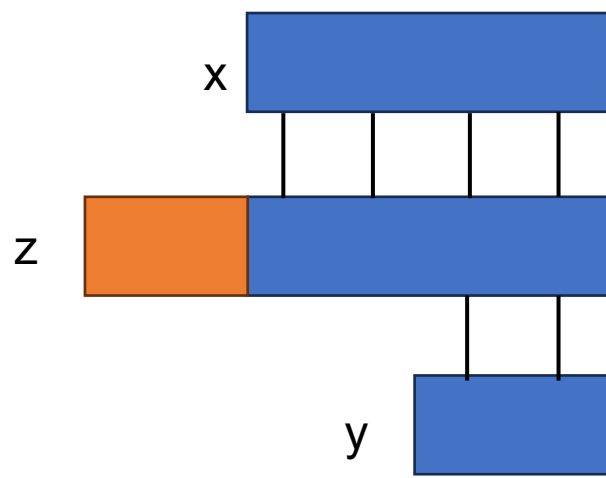
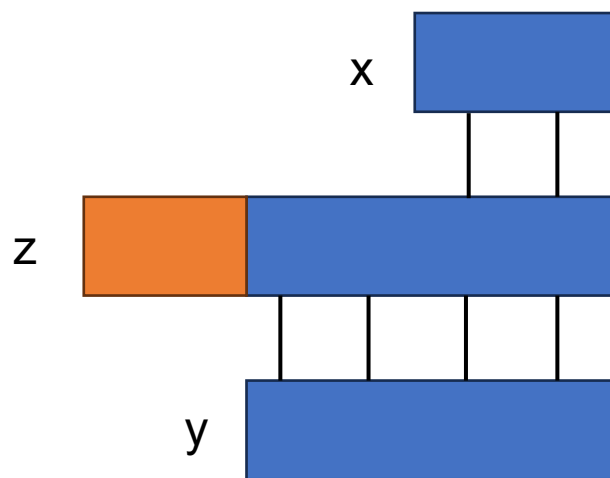
1. Пустая строка ε является одновременно и суффиксом, и префиксом любой строки.
2. Для любых строк x и y и для любого символа a соотношение $x \sqsupset y$ выполняется тогда и только тогда, когда $xa \sqsupset ya$.
3. Отношения $\sqsubset$ и $\sqsupset$ являются транзитивными.



Обозначения и терминология

Лемма 9.5 (Лемма о перекрывающихся суффиксах)

Предположим, что x , y и z являются строками, такими, что $x \supset z$ и $y \supset z$. Если $|x| \leq |y|$, то $x \supset y$. Если $|x| \geq |y|$, то $y \supset x$. Если $|x| = |y|$, то $x = y$.



Pattern matching

Разные метрики

- Левинштейна
- Конечный автомат

Разные алгоритмы:

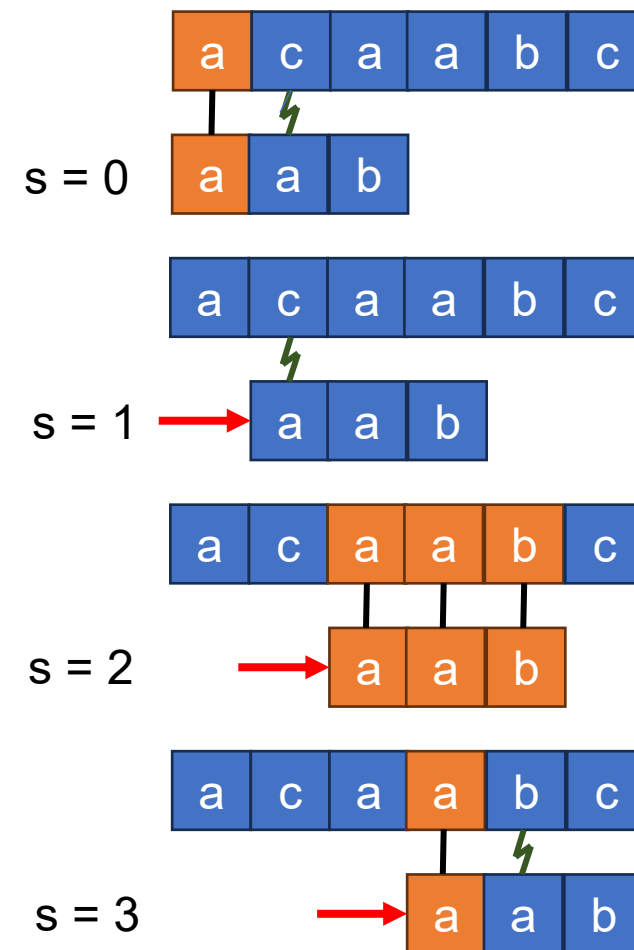
- Рабина-Карпа
- Кнута-Морриса-Пратта
- Бойера-Мура
- Алгоритм Ахо-Корасик



Простейший алгоритм поиска подстрок

```
void NaiveStringMatcher(const char *T, const char *P) {
    int n = strlen(T);
    int m = strlen(P);
    for (int s = 0; s <= n - m; s++) {
        int match = 1;
        for (int i = 0; i < m; i++) {
            if (P[i] != T[s + i]) {
                match = 0;
                break;
            }
        }
        if (match)
            printf("Образец найден со сдвигом %d\n", s);
    }
}
```

Время работы процедуры NaiveStringMatcher равно $O((n - m + 1)m)$, и это точная оценка в наихудшем случае.



Алгоритм Рабина-Карпа

- придумали Рабин (Rabin) и Карп (Karp)
- допускает обобщения на другие родственные задачи, такие как задача о сопоставлении двумерного образца.
- $\Theta(m)$ - на предварительная обработка
- $\Theta((n - m + 1)m)$ - время его работы в наихудшем случае
- среднее время работы этого алгоритма существенно лучше.

Для простоты **предположим**, что $\Sigma = \{0, 1, 2, \dots, 9\}$, т.е. что каждый символ представляет собой десятичную цифру. (В общем случае каждый символ — это цифра в системе счисления с основанием d , где $d = |\Sigma|$.)

Для заданного образца $P[1\dots m]$ - p соответствующее ему десятичное значение.

Для заданного текста $T[1\dots n]$ - t_s десятичное значение подстрок $T[s + 1 \dots s + m]$ длиной m при $s = 0, 1, \dots, n - m$.

Хотим: p - время $\Theta(m)$, t_s - $\Theta(n - m + 1)$, тогда значения всех допустимых сдвигов $\Theta(m) + \Theta(n - m + 1) = \Theta(n)$ путем сравнения значения p с каждым из значений t_s .



Алгоритм Рабина-Карпа

Правило Горнера - p за время $\Theta(m)$:

$$p = P[m] + 10 (P[m - 1] + 10(P[m - 2] + \dots + 10(P[2] + 10P[1])\dots)).$$

t_0 можно вычислить из $T[1\dots m]$ за то же время $\Theta(m)$.

Остальные значения $t_1, t_2, \dots, t_{n-m}$ за время $\Theta(n - m)$, заметим, что величину t_{s+1} можно вычислить из величины t_s за константное время, так как

$$t_{s+1} = 10(t_s - 10^{m-1}T[s + 1]) + T[s + m + 1].$$

Вычитание $10^{m-1}T[s + 1]$ удаляет старшую цифру из t_s , умножение полученного результата на 10 сдвигает число влево на один десятичный разряд, а добавление $T[s + m + 1]$ вносит в него соответствующую младшую цифру.

Пример

$m = 5$ и $t_s = 31415$, то необходимо удалить старшую цифру $T[s + 1] = 3$ и добавить новую младшую цифру (предположим, это $T[s + 5 + 1] = 2$), чтобы получить

$$t_{s+1} = 10(31415 - 10000 \cdot 3) + 2 = 14152.$$

Итог

p - $\Theta(m)$, величины $t_0, t_1, \dots, t_{n-m}$ - за время $\Theta(n - m + 1)$, а все вхождения образца $P[1\dots m]$ в текст $T[1\dots n]$ можно найти, затратив на фазу предварительной обработки время $\Theta(m)$, а на фазу сравнения - время $\Theta(n - m + 1)$.



Алгоритм Рабина-Карпа

Сложность: p и t_s могут оказаться слишком большими

Если образец P содержит m символов, то предположение о том, что каждая арифметическая операция с числом p (в которое входит m цифр) занимает "фиксированное время", не отвечает действительности.

Решение: вычислять значения p и t по модулю некоторого числа q .

$p \bmod q$ можно вычислить за время $\Theta(m)$,

$t \bmod q$ - за время $\Theta(n - m + 1)$

Выберем q такое простое число, что $10q$ помещается в компьютерное слово

Все необходимые вычисления можно выполнить с использованием арифметики одинарной точности.

В общем случае, если имеется d -символьный алфавит $\{0, 1, \dots, d - 1\}$, значение q выбирается таким образом, чтобы величина dq помещалась в компьютерное слово, и рекуррентное соотношение исправляется так, чтобы оно работало по модулю q , т.е. записываем его в виде

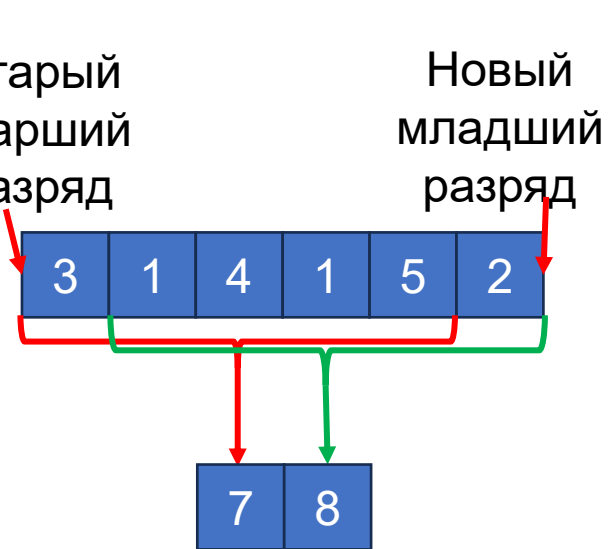
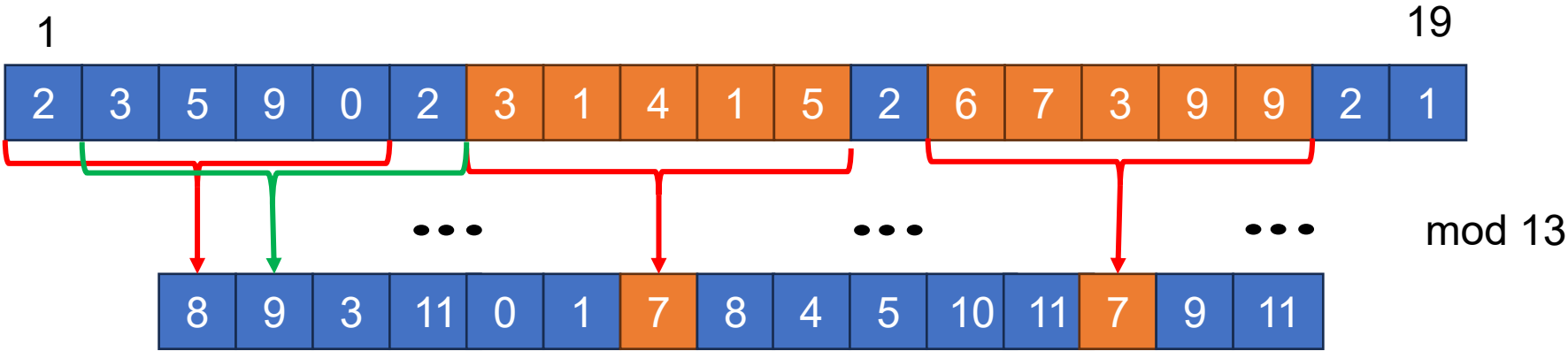
$$t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) \bmod q,$$

где $h = d^{m-1} \bmod q$ - значение цифры "1" в старшем разряде окна размером m цифр.



Алгоритм Рабина-Карпа

P = 31415



Истинное совпадение Ложное совпадение

Старый старший разряд Сдвиг Новый младший разряд

$$\begin{aligned} 14152 &\equiv (31415 - 10000 \cdot 3) \cdot 10 + 2 \pmod{13} \\ &\equiv (7 - 3 \cdot 3) \cdot 10 + 2 \pmod{13} \\ &\equiv 8 \pmod{13} \end{aligned}$$



Конечные автоматы (ФА)

Мотивация:

Фиксация алфавита

Пусть L – произвольное множество строк, тогда $L \subseteq \Sigma^*$

Как понять принадлежит ли строка $\alpha \in L$

Применения:

- Компиляторы
- Сетевые протоколы
- Численные методы для описания физических процессов
- Регулярные выражения

Пример с компилятором:

token

Имя переменной, литералы, комментарии, директивы препроцессора, функции и т. п.

Способом описания являются регулярные выражения.

Конечные автоматы являются одним из способов записи таких выражений



Регулярные выражения

Regex описывается с помощью некоторого языка.

Примеры:

" x " $\rightarrow \{x\}$

" $\emptyset$ " $\rightarrow \{\emptyset\}$

" ε " $\rightarrow \{\varepsilon\}$

Конкатенация: " $R_1 R_2$ " $\rightarrow \{\alpha\beta \mid \alpha \in L_1, \beta \in L_2\} = L_1 L_2$

Объединение языков: " $R_1 | R_2$ " $\rightarrow L_1 \cup L_2$

Язык бинарных последовательностей длины 3: $(0|1)(0|1)(0|1)$

Разбиение строки на множество подстрочек, описываемых регулярным выражением " R_1^* " $\rightarrow \{\varepsilon\} \cup L \cup LL \cup LLL \cup \dots$

$(0|1)^*$ $\rightarrow$ все последовательности 0 и 1 включая пустую

Можно придумывать новые операторы через текущие:

" $R?$ " $\rightarrow \varepsilon \mid R$

" R^+ " $\rightarrow RR^*$

$\alpha := _ | a | b | \dots | Z |$

$\text{digit} := 0 | 1 | \dots | 9 |$

$\text{id} := \alpha (\alpha | \text{digit})^*$

Пример id: `int ivan_peredkov_2007 = otchislen`



Конечные автоматы

$\Sigma = \{0,1,2\}$

$q_0 \in Q$ - *начальное состояние*

Какие строки попадают в язык?

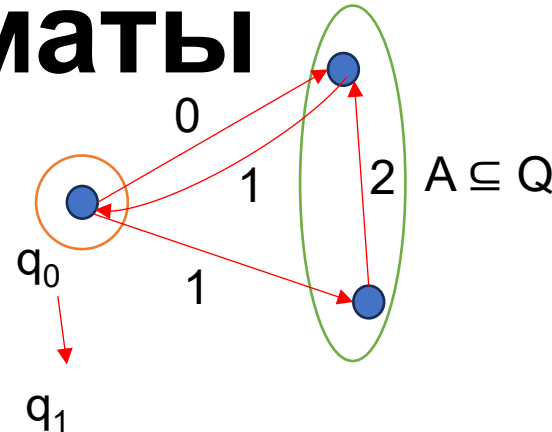
Начать с начального состояния, читать символ в строке и совершить переход по стрелке на которой он написан.

Если стрелки нет – строка плохая.

$q_1 \in A$ – множество допускающих состояний, тогда наша строка – язык

Примеры: $L = \{0, 01010, 1211, \dots\}$

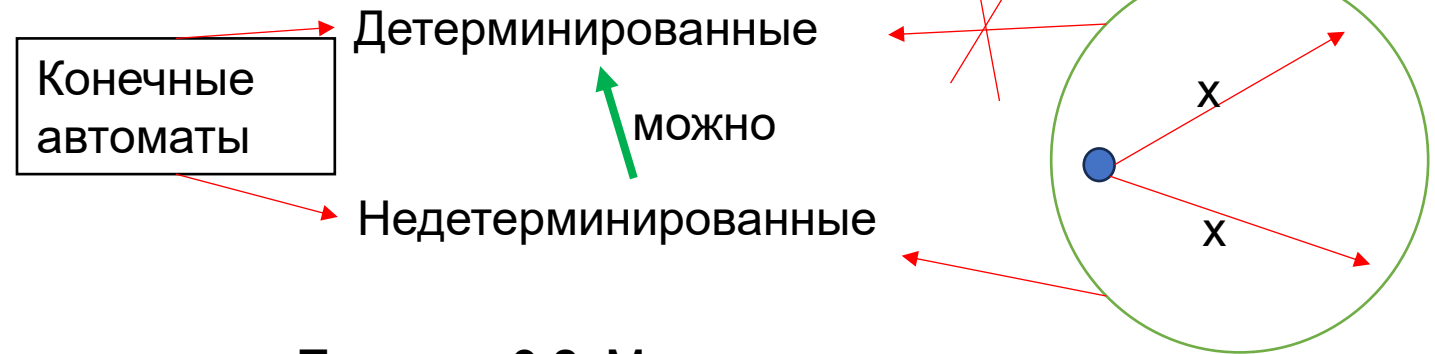
Регулярное выражение: $(01|121)^* (0|1|12)$



Теорема 9.1.

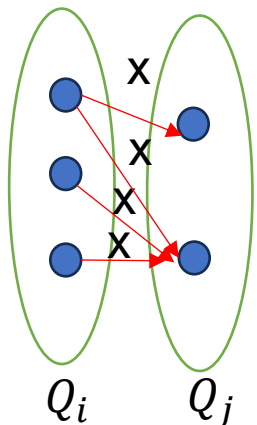
- 1) Если что-то можно описать регулярным выражением это можно описать конечным автоматом
- 2) Если что-то можно описать регулярным выражением это можно описать конечным автоматом

Без доказательства



$q_i \rightarrow q_j$
 $Q_i \rightarrow Q_j$

При проходе множества нужно сделать выбор в каждой точке состояний где он возможен. Важно чтобы 1 из них было терминальным



Теорема 9.2. Можно из недетерминированного в детерминированное.

$q_0^{DFA} = \{q_0^{NFA}\} \quad A^{DFA} = \{A_i \subseteq Q^{NFA} | A_i \cap A^{NFA} \neq \emptyset\}$

$A \rightarrow^x B$

Конечные автоматы

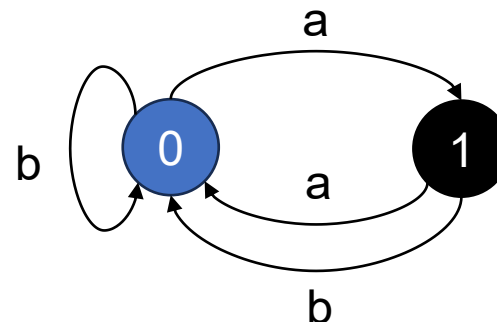
Конечный автомат M представляет собой кортеж из пяти значений $(Q, q_0, A, \Sigma, \delta)$, где

- Q - конечное множество состояний;
- $q_0 \in Q$ - **начальное состояние**;
- $A \subseteq Q$ - множество различных **допускающих состояний**;
- Σ - конечный входной алфавит;
- δ - функция, отображающая $Q \times \Sigma$ в Q и называемая **функцией переходов автомата M** .

С конечным автоматом M связана функция φ , которая называется **функцией конечного состояния** (final state function) и отображает множество Σ^* на множество Q , так, что значение $\varphi(\omega)$ представляет собой состояние, в котором оказывается автомат M после сканирования строки ω . Таким образом, M принимает строку w тогда и только тогда, когда $\varphi(w) \in A$. Функция φ определяется следующим рекуррентным соотношением с использованием функции переходов:

$$\begin{aligned} \varphi(\varepsilon) &= q_0 \\ \varphi(\omega a) &= \delta(\varphi(\omega), a) \text{ для } \omega \in \Sigma^*, a \in \Sigma. \end{aligned}$$

	Вход	
Состояние	a	b
0	1	0
1	0	0



Автоматы поиска подстрок

Предобработка: Для каждого образца P строится свой автомат поиска подстрок.

Предполагается: образец P — это заданная фиксированная строка: для краткости мы будем опускать в обозначениях зависимость от P .

Определим функцию σ : **суффиксная функция** (suffix function)

Функция σ является отображением множества Σ^* на множество $\{0, 1, \dots, m\}$, таким, что величина $\sigma(x)$ равна длине максимального префикса P , который является суффиксом строки x :

$$\sigma(x) = \max\{k: P_k \sqsupseteq x\}$$

Суффиксная функция всегда определена ($P_0 = \varepsilon$ — пустая строка).

Пример: образец $P = ab$; тогда $\sigma(\varepsilon) = 0$, $\sigma(\text{сacca}) = 1$ и $\sigma(\text{ссаb}) = 2$.

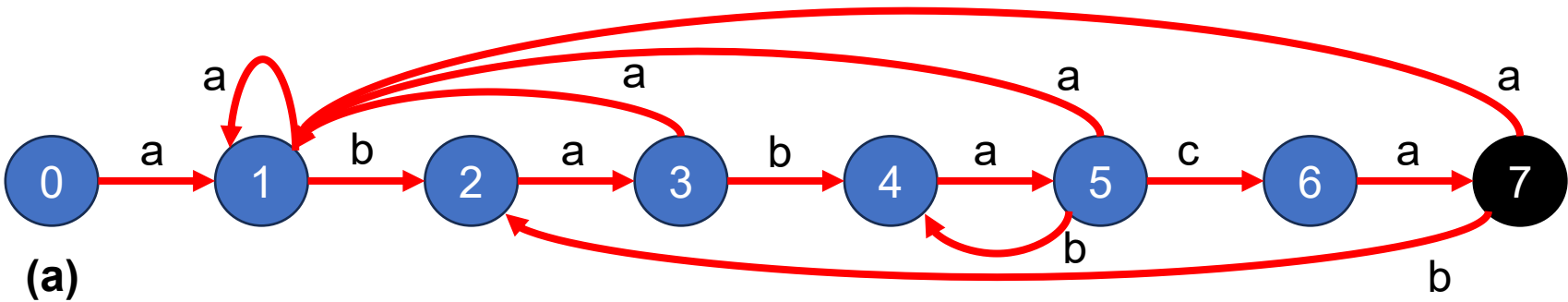
Для образца P длиной m равенство $\sigma(x) = m$ выполняется тогда и только тогда, когда $P \sqsupseteq x$. Из определения суффиксной функции следует, что если $x \sqsupseteq y$, то $\sigma(x) \leq \sigma(y)$.

Определим автомат поиска подстрок, соответствующий образцу $P[1\dots m]$:

- Множество состояний Q имеет вид $\{0, 1, \dots, m\}$. q_0 - состояние 0, m является единственным принимающим состоянием.
- Функция переходов δ определена для любого состояния q и символа a уравнением
$$\delta(q, a) = \sigma(P_q a).$$



Автоматы поиска подстрок



(a)

		Вход		
Состояние		a	b	c
0		1	0	0
1		1	2	0
2		3	0	0
3		1	4	0
4		5	0	0
5		1	4	6
6		7	0	0
7		1	2	0

(b)

(b)

i	-	1	2	3	4	5	6	7	8	9	10	11
T[i]	-	a	b	a	b	a	b	a	c	a	b	a
Состояние $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

- (a) Диаграмма состояний автомата поиска подстрок, который воспринимает все строки, оканчивающиеся строкой ababaca. Состояние 0 - начальное, а состояние 7 (оно выделено черным цветом) - единственное допускающее.
- (б) Соответствующая функция переходов δ , а также строка образца P = ababaca.
- (в) Обработка автоматом текста T = abababacaba. Под каждым символом текста T[i] указано состояние $\phi(T_i)$, в котором находится автомат после обработки префикса T_i.

Автоматы поиска подстрок

```
// Основная функция Finite Automaton Matcher
void FiniteAutomatonMatcher(char T[], char P[], int m) {
    int n = strlen(T); // Длина текста
    // Текущее состояние автомата
    for (int q = 0, i = 0; i < n; i++) {
        q = transitionFunction(q, T[i], P, m);
        if (q == m) {
            printf("Образец найден со сдвигом %d\n", i - m + 1);
        }
    }
}
```

Чтобы пояснить работу автомата, предназначенного для поиска подстрок, приведем простую, но эффективную программу для моделирования поведения такого автомата (представленного функцией переходов δ) при поиске образца P длиной m во входном тексте $T[1\dots n]$.

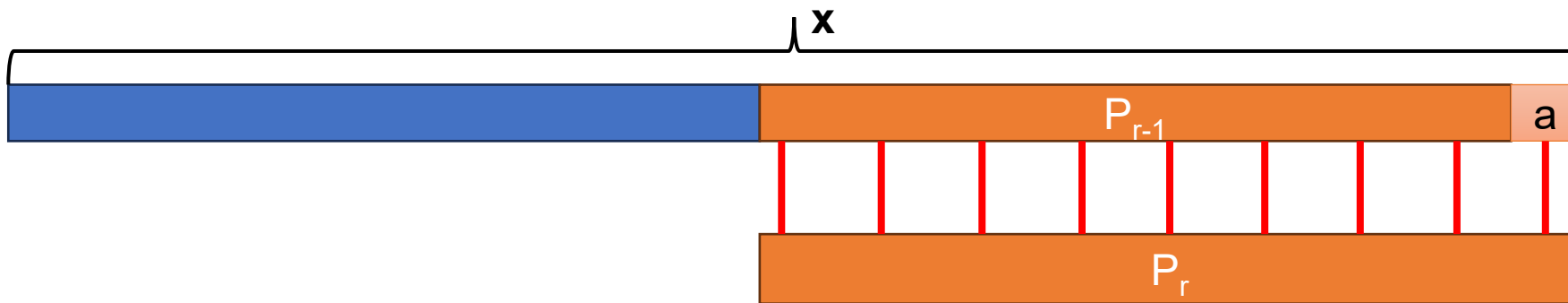
Из простой циклической структуры процедуры Finite-Automaton-Matcher можно легко увидеть, что время поиска совпадений с его помощью в тексте длиной n равно $\Theta(n)$. Однако сюда не входит время предварительной обработки, которое необходимо для вычисления функции переходов δ . К этой задаче мы обратимся позже, после доказательства корректности работы процедуры Finite-Automaton-Matcher. Рассмотрим, как автомат обрабатывает входной текст $T[1\dots n]$. Докажем, что после сканирования символа $T[i]$ автомат окажется в состоянии $\sigma(T_i)$. Поскольку соотношение $\sigma(T_i) = m$ справедливо тогда и только тогда, когда $P \sqsupseteq T_i$, машина окажется в принимающем состоянии m тогда и только тогда, когда она только что считает образец P . Чтобы доказать этот результат, воспользуемся двумя приведенными ниже леммами, в которых идет речь о суффиксной функции σ .

Автоматы поиска подстрок

Лемма 9.6 (Неравенство суффиксной функции)

Для любой строки x и символа a выполняется неравенство $\sigma(xa) \leq \sigma(x) + 1$.

Доказательство. Введем обозначение $r = \sigma(xa)$ (рисунок). Если $r = 0$, то неравенство $\sigma(xa) = r \leq \sigma(x) + 1$ тривиальным образом следует из того, что величина $\sigma(x)$ неотрицательна. Теперь предположим, что $r > 0$. Тогда $P_r \supset xa$ согласно определению функции σ . Если в конце строк P_r и xa отбросить по символу a , то получим $P_{r-1} \supset x$. Следовательно, справедливо неравенство $r - 1 \leq \sigma(x)$, так как $\sigma(x)$ - наибольшее значение k , при котором выполняется соотношение $P_k \supset x$, и, таким образом, $\sigma(xa) = r \leq \sigma(x) + 1$.



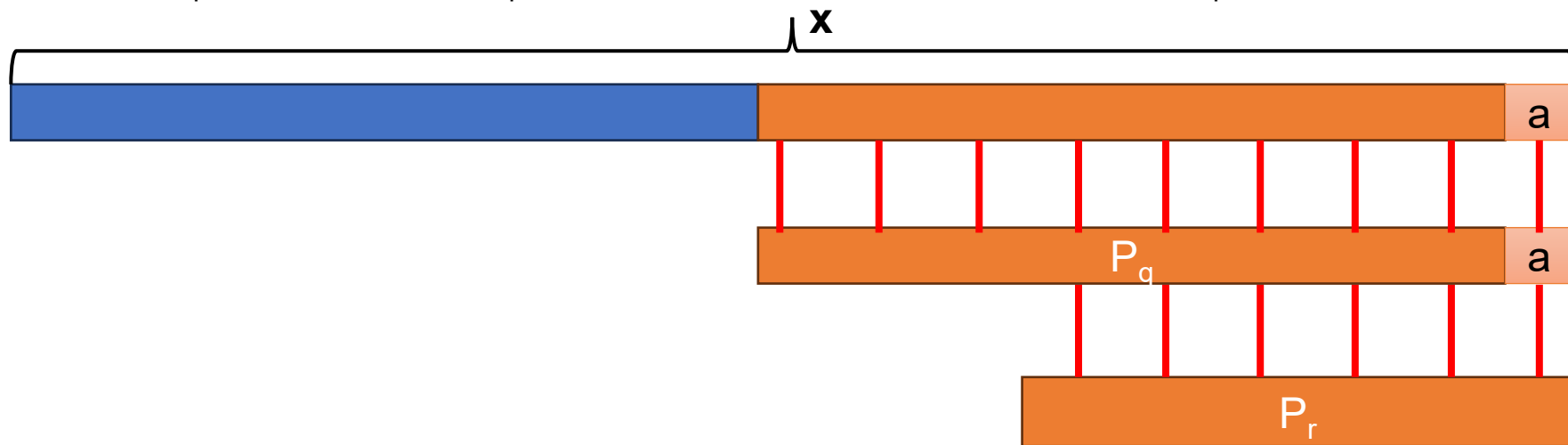
На рисунке показано, что $r \leq \sigma(x) + 1$, где $r = \sigma(xa)$.

Автоматы поиска подстрок

Лемма 9.7 (Лемма о рекурсии суффиксной функции)

Если для произвольной строки x и символа a выполняется $q = \sigma(x)$, то $\sigma(xa) = \sigma(P_q a)$.

Доказательство. $P_q \supset x$ из определения σ . Как показано на рисунке, выполняется также $P_q a \supset xa$. Если обозначить $r = \sigma(xa)$, то $P_r \supset xa$ и, согласно лемме 2.2, $r \leq q + 1$. Таким образом, имеем $|P_r| = r \leq q + 1 = |P_q a|$. Поскольку $P_q a \supset xa$, $P_r \supset xa$ и $|P_r| \leq |P_q a|$, из леммы 2.1 следует, что $P_r \supset P_q a$. Следовательно, $r \leq \sigma(P_q a)$, т.е. $\sigma(xa) \leq \sigma(P_q a)$. Но, кроме того, $\sigma(xa) \geq \sigma(P_q a)$, поскольку $P_q a \supset xa$. Таким образом, $\sigma(xa) = \sigma(P_q a)$.



На рисунке показано, что $r = \sigma(P_q a)$, где $q = \sigma(x)$ и $r = \sigma(xa)$.

Автоматы поиска подстрок

Основная теорема, характеризующая поведение автомата поиска подстрок при обработке входного текста.

Теорема 9.3

Если φ - функция конечного состояния автомата поиска подстрок для заданного образца P , а $T[1...n]$ - входной текст автомата, то $\varphi(T_i) = \sigma(T_i)$

Доказательство. Доказательство выполняется по индукции по i . Для $i = 0$ теорема тривиально истинна, поскольку $T_0 = \varepsilon$. Таким образом, $\varphi(T_0) = 0 = \sigma(T_0)$.

Теперь предположим, что $\varphi(T_i) = \sigma(T_i)$ и докажем, что $\varphi(T_{i+1}) = \sigma(T_{i+1})$. Обозначим $\varphi(T_i)$ как q , $T[i + 1]$ - как a . Тогда

$$\begin{aligned}\varphi(T_{i+1}) &= \varphi(T_i a) \text{ (согласно определению } T_{i+1} \text{ и } a) \\ &= \delta(\varphi(T_i), a) \text{ (согласно определению } \varphi) \\ &= \delta(q, a) \text{ (согласно определению } q) \\ &= \sigma(P_q a) \text{ (согласно определению } \delta) \\ &= \sigma(T_i a) \text{ (согласно лемме 9.7 и гипотезе индукции)} \\ &= \sigma(T_{i+1}) \text{ (согласно определению } T_{i+1}).\end{aligned}$$

Вычисление функции переходов

```
// Вычисление функции переходов
int **ComputeTransitionFunction(char *P, char *Sigma) {
    int m = strlen(P);           // Длина образца
    int sigmaSize = strlen(Sigma); // Размер алфавита
    int q, i, k;
    // Выделение памяти под таблицу переходов  $\delta[q][a]$ 
    int **delta = (int **)malloc((m + 1) * sizeof(int *));
    for (q = 0; q <= m; q++)
        delta[q] = (int *)malloc(sigmaSize * sizeof(int));
    // Заполнение таблицы переходов
    for (q = 0; q <= m; q++) {
        for (i = 0; i < sigmaSize; i++) {
            char a = Sigma[i];
            k = (m + 1 < q + 2) ? m + 1 : q + 2; //  $\min(m + 1, q + 2)$ 
            do {
                k--;
            } while (!isSuffix(P, k, q, a));
            delta[q][i] = k;
        }
    }
    return delta;
}
```

В этой процедуре функция $\delta(q, a)$ вычисляется непосредственно, исходя из ее определения. Время работы процедуры Compute-Transition-Function равно $O(m^3|\Sigma|)$, поскольку внешние циклы дают вклад, соответствующий умножению на $m|\Sigma|$, внутренний цикл repeat может повториться не более $m + 1$ раз, а для выполнения проверки $P_k \supseteq P_q a$ может потребоваться сравнить вплоть до m символов.

Алгоритм Кнута-Морриса-Пратта

Алгоритм сравнения строк, который был предложено Кнутом (Knuth), Моррисом (Morris) и Праттом (Pratt).

Избежим вычисление функции переходов δ .

Используем вспомогательную функцию π , которая вычисляется по заданному образцу за время $\Theta(m)$ и хранится в массиве $\pi[1...m]$, время сравнения в этом алгоритме оказывается равным $\Theta(n)$. Массив π позволяет эффективно (в амортизированном смысле) вычислять функцию δ "на лету", т.е. по мере необходимости.

Для любого состояния $q = 0, 1, \dots, m$ и любого символа $a \in \Sigma$ величина $\pi[q]$ содержит информацию, необходимую для вычисления величины $\delta(q, a)$, но не зависит от a .

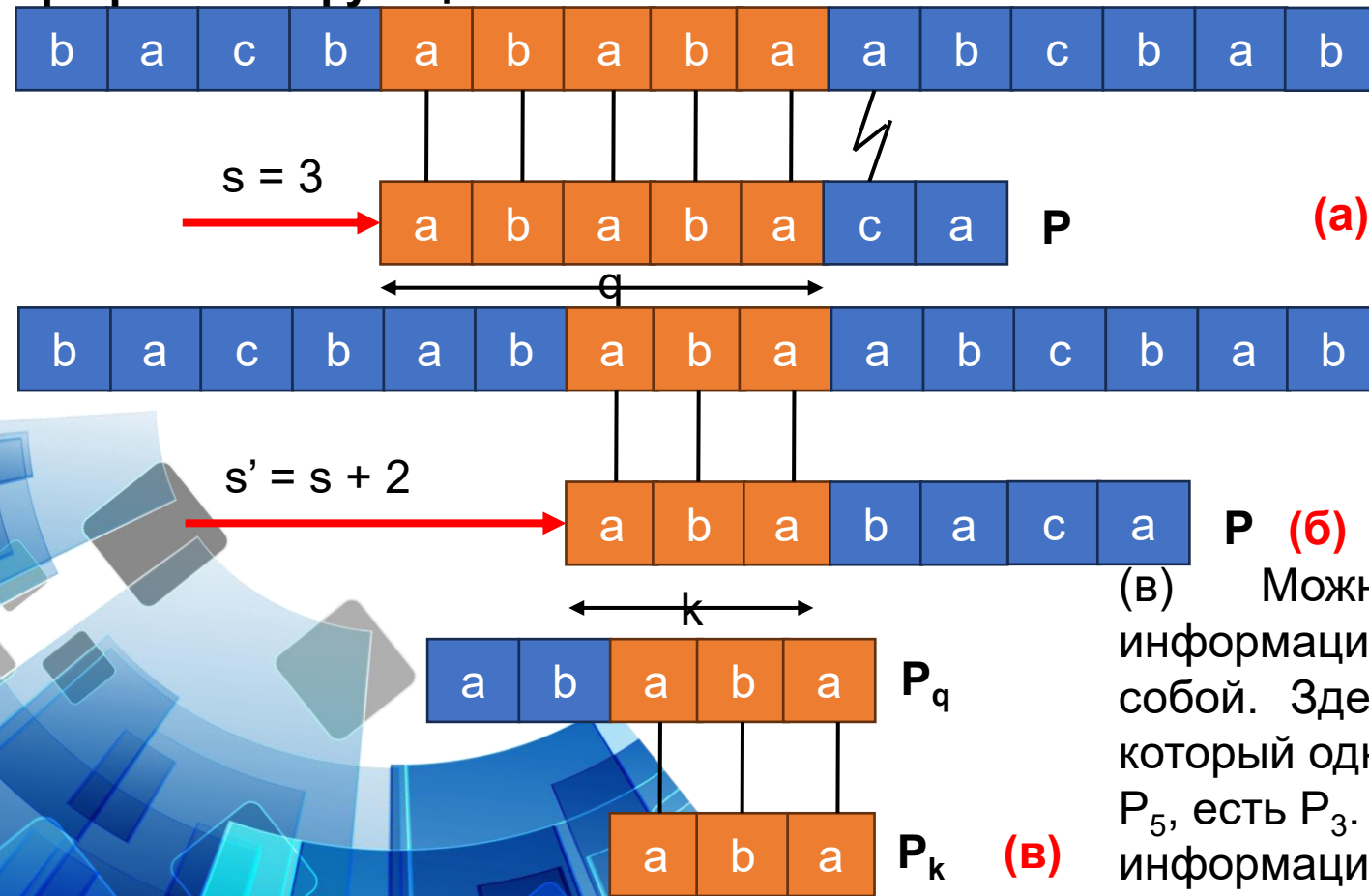
Поскольку массив π содержит только m элементов (в то время как в массиве δ их $O(m|\Sigma|)$), вычисляя на этапе предварительной обработки функцию π вместо функции δ , удастся уменьшить время предварительной обработки образца в $|\Sigma|$ раз.



Алгоритм Кнута-Морриса-Пратта

Префиксная функция π для некоторого образца инкапсулирует сведения о том, в какой мере образец совпадает сам с собой после сдвигов. Эта информация позволяет избежать ненужных проверок в простейшем алгоритме поиска подстрок и предвычисления функции δ . $\pi = \varphi(P, \pi, k)$

Префиксная функция π .



(а) Образец $P = ababasa$, сдвинутый относительно T так, что совпадают первые $q = 5$ символов. Совпадающие символы выделены серой штриховкой и соединены вертикальными линиями.

(б) Пользуясь одной лишь информацией о пяти совпадающих символах, можно вывести, что сдвиг $s + 1$ недопустим, но сдвиг $s' = s + 2$ согласуется со всем, что мы знаем о тексте, а значит, потенциально допустим.

(в) Можно предварительно вычислить полезную информацию для таких выводов, сравнив образец с самим собой. Здесь мы видим, что наидлиннейший префикс P , который одновременно является истинным суффиксом P_5 , есть P_3 . Представим эту предвычисленную информацию в массиве π , так что $\pi[5] = 3$.

Алгоритм Кнута-Морриса-Пратта

Известно, что символы $P[1...q]$ образца соответствуют символам $T[s + 1... s + q]$ текста.

Каков наименьший сдвиг $s' > s$, такой, что для некоторого $k < q$

$P[1...k] = T[s' + 1...s' + k]$,

где $s' + k = s + q$? Грубо $\min \pi = \varphi(P, q)$

s – начало мэтчинга – растет со сдвигом

q – длина мэтчинга – растет если не растет сдвиг

$s := s + \varphi(q)$; $q := q - \varphi(q)$, $\varphi(q)$ – сдвиг

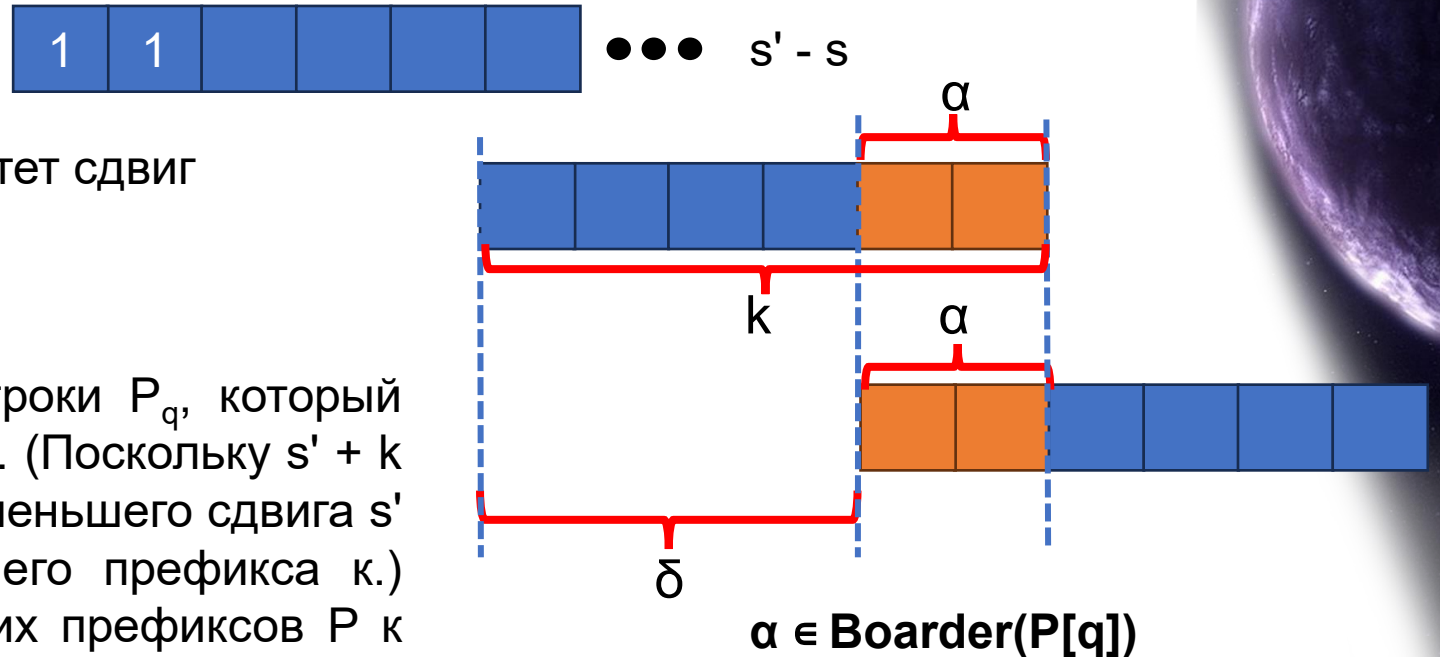
Boarder $(\alpha) = \{\lambda \mid \lambda \sqsubset \alpha, \lambda \sqsupset \alpha\}$

Пример: **Boarder**(abba) = { ϵ , a, abba}

Необходимо найти $\max$ префикс P_k строки P_q , который одновременно является суффиксом T_{s+q} . (Поскольку $s' + k = s + q$, если заданы s и q , то поиск наименьшего сдвига s' эквивалентен поиску длины наибольшего префикса k .)

Мы добавляем разность $q - k$ длин этих префиксов P_k к сдвигу s , чтобы получить новый сдвиг s' , что $s' = s + (q - k)$. В лучшем случае $k = 0$, так что $s' = s + q$ и мы тут же пропускаем сдвиги $s + 1, s + 2, \dots, s + q - 1$. В любом случае при новом сдвиге s' не нужно сравнивать первые k символов P с соответствующими символами T .

Необходимую информацию можно вычислить путем сравнения образца с самим собой.



Формализуем предварительно вычисляемую информацию следующим образом. Для заданного образца $P[1...m]$ префиксной функцией для P является функция π : $\{1, 2, \dots, m\} \rightarrow \{0, 1, \dots, m - 1\}$, такая, что $\pi[q] = \max\{k : k < q \text{ и } P_k \sqsupset P_q\}$

Корректность вычисления префиксной функции

Лемма 9.8 (Лемма об итерации префиксной функции)

Пусть P - образец длиной m с префиксной функцией π . Тогда для $q = 1, 2, \dots, m$ имеем $\pi^*[q] = \{k : k < q \text{ и } P_k \supset P_q\}$.

Доказательство. Сначала докажем, что $\pi^*[q] \subseteq \{k : k < q \text{ и } P_k \supset P_q\}$ или, что эквивалентно, из $i \in \pi^*[q]$ следует $P_i \supset P_q$.

Если $i \in \pi^*[q]$, то $i = \pi^{(u)}[q]$ для некоторого $u > 0$. Докажем уравнение по индукции по u . Для $u = 1$ имеем $i = \pi[q]$, и наше утверждение следует из того, что $i < q$ и $P_{\pi[q]} \supset P_q$ по определению π . Используя отношения $\pi[i] < i$ и $P_{\pi[i]} \supset P_i$ и транзитивность $<$ и $\supset$, устанавливаем справедливость утверждения для всех i из $\pi^*[q]$. Следовательно, $\pi^*[q] \subseteq \{k : k < q \text{ и } P_k \supset P_q\}$.

Теперь докажем, что $\{k : k < q \text{ и } P_k \supset P_q\} \subseteq \pi^*[q]$, методом от противного. Предположим, что множество $\{k : k < q \text{ и } P_k \supset P_q\} - \pi^*[q]$ непустое, и пусть j представляет собой наибольшее число этого множества. Поскольку $\pi[q]$ является наибольшим значением в $\{k : k < q \text{ и } P_k \supset P_q\}$ и $\pi[q] \in \pi^*[q]$, должно выполняться $j < \pi[q]$, так что пусть j' обозначает наименьшее целое из $\pi^*[q]$, большее, чем j . (Можно выбрать $j' = \pi[q]$, если никакие другие числа в $\pi^*[q]$ не превышают j .) Имеем $P_j \supset P_q$, поскольку $j \in \{k : k < q \text{ и } P_k \supset P_q\}$, и из $j' \in \pi^*[q]$ и уравнения теоремы получаем $P_{j'} \supset P_q$. Таким образом, $P_j \supset P_{j'}$ согласно лемме 9.5, и j является наибольшим значением, меньшим j' и обладающим этим свойством. Следовательно, должны выполняться $\pi^*[j'] = j$ и, поскольку $j' \in \pi^*[q]$, $j \in \pi^*[q]$. Это противоречие и доказывает лемму.

Положим

$$\pi^*[q] = \{\pi[q], \pi^{(2)}[q], \pi^{(3)}[q], \dots, \pi^{(t)}[q]\},$$

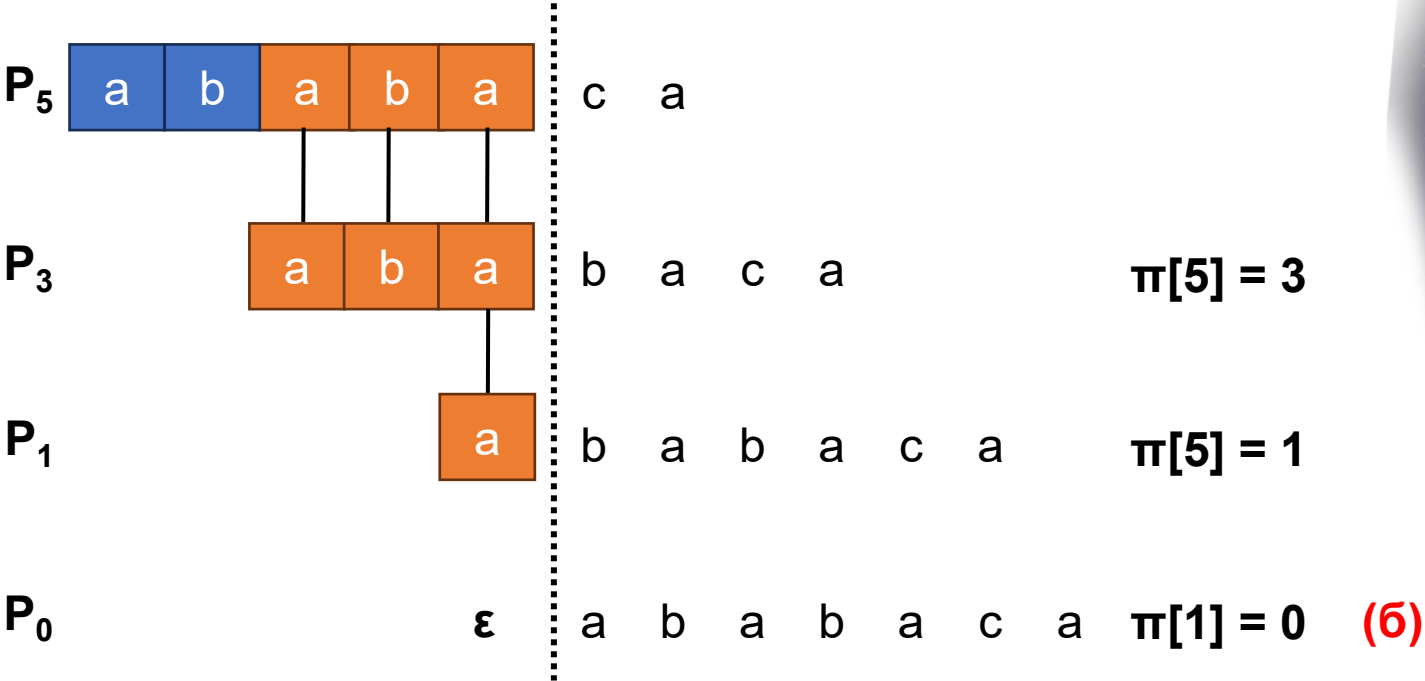
где $\pi^{(i)}[q]$ определена в терминах функциональной итерации, так что $\pi^{(0)}[q] = q$ и $\pi^{(i)}[q] = \pi[\pi^{(i-1)}[q]]$ для $i > 1$, и где последовательность $\pi^*[q]$ останавливается по достижении $\pi^{(t)}[q] = 0$.

Алгоритм Кнута-Морриса-Пратта

i	1	2	3	4	5	6	7
P[i]	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

(a)

Иллюстрация леммы 9.8 для образца $P = ababasa$ и $q = 5$.
(a) Функция π для указанного образца. Поскольку $\pi[5] = 3$, $\pi[3] = 1$ и $\pi[1] = 0$, итерируя π , получаем $\pi^*[5] = \{3, 1, 0\}$.



(б) Мы смещаем образец P вправо и замечаем, когда некоторый префикс P_k образца P соответствует некоторому истинному суффиксу P_5 ; мы получаем соответствие при $k = 3, 1$ и 0 . На рисунке в первой строке приведен образец P , а пунктирная вертикальная линия проведена после P_5 . Последовательные строки показывают все сдвиги P , которые приводят к тому, что некоторый префикс P_k строки P соответствует некоторому суффиксу P_5 . Совпадающие символы выделены штриховкой и соединены вертикальными линиями. Таким образом, $\{k: k < 5 \text{ и } P_k \supseteq P_5\} = \{3, 1, 0\}$. Лемма 9.8 утверждает, что $\pi^*[5] = \{k : k < q \text{ и } P_k \supseteq P_q\}$ для всех q .

Алгоритм Кнута-Морриса-Пратта

```
// Функция для вычисления префикс-функции (Compute-Prefix-Function)
int *Compute_Prefix_Function(char *P) {
    int m = strlen(P); // Длина образца
    int *pi = (int *)malloc(m * sizeof(int)); // Массив для префикс-функции (индекс с 1)
    pi[0] = 0; // Первый символ не имеет префиксов
    int k = 0; // Длина текущего префикса
    for (int q = 1; q < m; q++) {
        while (k > 0 && P[k] != P[q - 1]) //Сравниваем символы (индексация с 0, поэтому P[q-1])
            k = pi[k]; // Переходим к меньшему префиксу
        if (P[k] == P[q - 1]) // Если символы совпадают
            k = k + 1; // Увеличиваем длину префикса
        pi[q] = k; // Сохраняем длину префикса для позиции q
    }
    return pi;
}
```

С помощью методов группового анализа можно показать, что время работы процедуры Compute-Prefix-Function равно $\Theta(m)$. Единственной тонкостью при этом является показ того, что всего цикл while выполняется $O(m)$ раз. Общее количество итераций цикла while не превышает $m - 1$, и время работы процедуры Compute-Prefix-Function составляет $\Theta(m)$.

Алгоритм Кнута-Морриса-Пратта

```
// Функция KMP для поиска образца в тексте
void KMP_Matcher(char *T, char *P) {
    int n = strlen(T);           // Длина текста
    int m = strlen(P);           // Длина образца
    int *pi = Compute_Prefix_Function(P); // Вычисляем префикс-функцию
    int q = 0;                   // Количество совпадающих символов
    for (int i = 1; i <= n; i++) { // Сканируем текст слева направо (индексация с 1)
        while (q > 0 && P[q] != T[i - 1]) // Пока q > 0 и символы не совпадают
            q = pi[q]; // Переходим к предыдущему возможному совпадению
        if (P[q] == T[i - 1]) // Если символы совпадают
            q = q + 1; // Увеличиваем количество совпадающих символов
        if (q == m) { // Если совпал весь образец
            printf("Образец находится со смещением %d\n", i - m); // Выводим позицию
            q = pi[q]; // Ищем следующее вхождение
        }
    }
    free(pi); // Освобождаем память
}
```

Можно показать с помощью группового анализа, что время сравнения процедуры KMP-Mather равно $\Theta(n)$.

Благодаря использованию функции π вместо функции δ , которая используется в процедуре Finite-Automaton-Matcher, время предварительной обработки образца уменьшается с $O(m|\Sigma|)$ до $\Theta(m)$, при том что время фактического сравнения остается ограниченным $\Theta(n)$.

Корректность вычисления префиксной функции

Лемма 9.9

Пусть P является образцом длиной m и пусть π представляет собой префиксную функцию для P . Если для $q = 1, 2, \dots, m$ выполняется $\pi[q] > 0$, то $\pi[q] - 1 \in \pi^*[q - 1]$.

Доказательство. Пусть $r = \pi[q] > 0$, так что $r < q$ и $P_r \supset P_q$; таким образом, $r-1 < q-1$ и $P_{r-1} \supset P_{q-1}$ (отбрасывая последние символы из P_r и P_q , что можно сделать, поскольку $r > 0$). Таким образом, согласно лемме 2.5 $r - 1 \in \pi^*[q - 1]$. А значит, $\pi[q] - 1 = r - 1 \in \pi^*[q - 1]$.

Определим для $q = 2, 3, \dots, m$ подмножество $E_{q-1} \subseteq \pi^*[q-1]$ как

$$\begin{aligned} E_{q-1} &= \{k \in \pi^*[q - 1] : P[k + 1] = P[q]\} \\ &= \{k : k < q - 1 \text{ и } P_k \supset P_{q-1} \text{ и } P[k + 1] = P[q]\} \text{ (из леммы 9.8)} \\ &= \{k : k < q - 1 \text{ и } P_{k+1} \supset P_q\} \end{aligned}$$

Множество E_{q-1} состоит из значений $k < q - 1$, для которых $P_k \supset P_{q-1}$ и для которых имеем $P_{k+1} \supset P_q$, поскольку $P[k + 1] = P[q]$. Таким образом, E_{q-1} состоит из значений $k \in \pi^*[q - 1]$, таких, что можно расширить P_k до P_{k+1} и получить истинный суффикс P_q .

Корректность вычисления префиксной функции

Следствие 9.2

Пусть P является образцом длиной m и пусть π представляет собой префиксную функцию для P . Для $q = 2, 3, \dots, m$

$$\pi[q] = \begin{cases} 0, & \text{если } E_{q-1} = \emptyset, \\ 1 + \max\{k \in E_{q-1}\}, & \text{если } E_{q-1} \neq \emptyset. \end{cases}$$

Доказательство. Если E_{q-1} - пустое множество, не существует $k \in \pi^*[q-1]$ (включая $k=0$), для которого можно расширить P_k до P_{k+1} и получить истинный префикс P_q . Следовательно, $\pi[q] = 0$.

Если множество E_{q-1} непустое, то для каждого $k \in E_{q-1}$ имеем $k+1 < q$ и $P_{k+1} \supset P_q$. Следовательно, из определения $\pi[q]$ имеем

$$\pi[q] \geq 1 + \max\{k \in E_{q-1}\}.$$

Заметим, что $\pi[q] > 0$. Пусть $r = \pi[q] - 1$, так что $r+1 = \pi[q]$ и, следовательно, $P_{r+1} \supset P_q$. Поскольку $r+1 > 0$, имеем $P[r+1] = P[q]$. Кроме того, согласно лемме 9.8 имеем $r \in \pi^*[q-1]$. Следовательно, $r \in E_{q-1}$, и, таким образом, $r \leq \max\{k \in E_{q-1}\}$, или, что эквивалентно,

$$\pi[q] \leq 1 + \max\{k \in E_{q-1}\}.$$

Объединение уравнений $\pi[q]$ завершает доказательство.

Корректность алгоритма Кнута-Морриса-Пратта

Процедуру KMP-Matcher можно рассматривать как видоизмененную реализацию процедуры Finite-Automaton-Matcher, использующую префиксную функцию π для вычисления переходов между состояниями.

Домашнее задание!



Для 12 задачи

z-функция

$z_i(\alpha) = |\text{lcp}(\alpha, \alpha[i,:])|$ - longest common prefix

a	b	a	c	a	b	a
7	0	1	0	3	0	1

α



z блок для $i > 0$, для $i = 0$ все 0



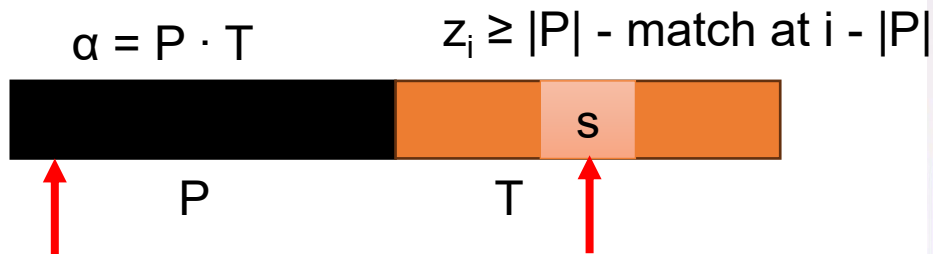
$k = i - j$

$\alpha[j, j + z_i]$

1. $z_i < z_j - k \rightarrow z_i = z_k$
2. $z_k \geq z_j - k \rightarrow z_i \geq z_j - k \rightarrow j = k$

Тоже работает за $O(n)$ поиск в строке, однако решение не очевидно. Происходит решение за счет введения z-функции

Мотивация



Как воспользоваться с опечаткой? Расстояние Левинштейна не более 1
1 замена / 1 вставка / 1 удаление

Для 13 задачи

Aho-Corasick

Есть $P_1 \dots P_k$ – паттерны(образцы). Мы хотим найти все паттерны в тексте за время $O(T + \sum_i P_i)$

